

```
--- djangoapi\core\testing\test_knoxLogin.py ---
```

```
from django.urls import reverse
from django.contrib.auth.models import User
from rest_framework.test import APITestCase
from rest_framework import status
from rest_framework.exceptions import ErrorDetail
```

```
#To manage to pass the test any integer or string
from core.myLib.generalModule import AnyInt, AnyStr
"""
```

```
DRF status codes
```

```
status.HTTP_200_OK
status.HTTP_201_CREATED
status.HTTP_202_ACCEPTED
status.HTTP_204_NO_CONTENT
status.HTTP_400_BAD_REQUEST
status.HTTP_401_UNAUTHORIZED
status.HTTP_402_PAYMENT_REQUIRED
status.HTTP_403_FORBIDDEN
status.HTTP_404_NOT_FOUND
status.HTTP_405_METHOD_NOT_ALLOWED
status.HTTP_500_INTERNAL_SERVER_ERROR
"""
```

```
from core.myLib.manageUsers import createUser
```

```
class KnoxLoginViewTest(APITestCase):
    def setUp(self):
```

```

                                                    self.admin_user =
createUser(username='admin2',password='admin2',email='admin2@gmail.com', is_active=True,
is_superuser=True)
        # Creamos un usuario normal que será al que le cambiaremos el estado
                                                    self.normal_user =
createUser(username='user1',password='123456789',email='user1@gmail.com',
is_active=True, is_superuser=False)
```

```
    def test_login_wrong_user_password(self):
        url = reverse('knox_login')
        d={'username': 'unexisting','password': 'wrong_password'}
        response = self.client.post(url, d, format='json')

        print("-----TEST 1-----")

print('core.testing.test_knoxLogin.KnoxLoginViewTest.test_login_wrong_user_password')
    print(f'Request url: {url}')
    print(f'Method: POST')
    print(f'Sent data: {d}')
    print(f'Response: {response}')
    print(f'Response status: {response.status_code}')
        print(f'json_data: {response.data}')#{'messages': {'error_request':
ErrorDetail(string='Wrong user or password.', code='invalid')}, 'politica_acceso':
```

```

{'acceso': 'Acceso abierto a la vista'}, 'data': None}

# Verification tests
self.assertEqual(response.status_code, 401)
self.assertEqual(response.data, {'messages': {'error_request':
ErrorDetail(string='Wrong user or password.', code='invalid')}, 'politica_acceso':
{'acceso': 'Acceso abierto a la vista'}, 'data': None})

def test_login_correct_user_password(self):
    url = reverse('knox_login')
    d={'username': 'admin2','password': 'admin2'}
    response = self.client.post(url, d, format='json')

    print("-----TEST 2-----")

print('core.testing.test_knoxLogin.KnoxLoginViewTest.test_login_correct_user_password')
print(f'Request url: {url}')
print(f'Method: POST')
print(f'Sent data: {d}')
print(f'Response: {response}')
print(f'Response status: {response.status_code}')
print(f'json_data: {response.data}')#{'messages': {'exito': 'Success
identification', 'serializer_message': 'Session created properly'}, 'politica_acceso':
{'acceso': 'Acceso abierto a la vista'}, 'data': [{'groups': [], 'username': 'admin2',
'opened_sessions': 1, 'user': 15, 'token':
'f7db5287110164af6cba57e928a2cec7878de1e473bf0fd818d70c4c5a4e4611', 'token_expiry':
datetime.datetime(2026, 5, 30, 9, 57, 5, 74159, tzinfo=datetime.timezone.utc)}}}

# Verification tests
self.assertEqual(response.status_code, 200)
self.assertEqual(response.data['messages'], {'exito': 'Success identification',
'serializer_message': 'Session created properly'})

def test_is_valid_token_correct_token(self):
    print("-----TEST 3-----")

print('core.testing.test_knoxLogin.KnoxLoginViewTest.test_is_valid_token_correct_token')
url = reverse('knox_login')
d={'username': 'admin2','password': 'admin2'}
response = self.client.post(url, d, format='json')
token=response.data["data"][0]["token"]
print(f"Token {token}")
url = reverse('is_valid_token')

response=self.client.post(url,{}, headers = {'Authorization':'Token ' + token})
print(f'Request url: {url}')
print(f'Method: POST')
print(f'Sent data: Data empty, token in the headers')
print(f'Response: {response}')
print(f'Response status: {response.status_code}')
print(f'json_data: {response.data}')#{'messages': {'exito': 'Identificación
realizada con éxito'}, 'politica_acceso': {'acceso': 'Acceso permitido.'}, 'data':
[{'detail': 'Token Válido.', 'username': 'admin2', 'user': 19, 'groups': [],

```

```
'opened_sessions': 1}}]
```

```
# Verification tests
```

```
# AnyInt() sirve para que pase el test cualquier entero
```

```
self.assertEqual(response.status_code, 200)
```

```
self.assertEqual(response.data, {'messages': {'exito': 'Identificación realizada  
con éxito'}, 'politica_acceso': {'acceso': 'Acceso permitido.'}, 'data': [{'detail':  
'Token Válido.', 'username': 'admin2', 'user': AnyInt(), 'groups': [],  
'opened_sessions': 1}]})
```

```
#Other way of authenticate users faster, avoiding post to the login endpoint
```

```
self.client.force_authenticate(user=self.admin_user)
```

```
#as the user is already authenticated, the Authentication header is set
```

```
response=self.client.post(url,{})
```

```
print("----TEST WITH FORCE AUTHENTICATED-----")
```

```
print(f'Request url: {url}')
```

```
print(f'Method: POST')
```

```
print(f'Sent data: Data empty, token in the headers')
```

```
print(f'Response: {response}')
```

```
print(f'Response status: {response.status_code}')
```

```
print(f'json_data: {response.data}')#{'messages': {'exito': 'Identificación  
realizada con éxito'}, 'politica_acceso': {'acceso': 'Acceso permitido.'}, 'data':  
[{'detail': 'Token Válido.', 'username': 'admin2', 'user': 19, 'groups': [],  
'opened_sessions': 1}]}
```

```
# Verification tests
```

```
# AnyInt() sirve para que pase el test cualquier entero
```

```
self.assertEqual(response.status_code, 200)
```

```
self.assertEqual(response.data, {'messages': {'exito': 'Identificación realizada  
con éxito'}, 'politica_acceso': {'acceso': 'Acceso permitido.'}, 'data': [{'detail':  
'Token Válido.', 'username': 'admin2', 'user': AnyInt(), 'groups': [],  
'opened_sessions': 1}]})
```